Introduktion till C++

Uppgift 4.3 – Diablo: Hell Factory

Uppgiftens målsättning
Studenten ska få en introduktion till *Factory* och *Type object patterns* samt röra sig mot mer datadriven utveckling.

Uppgift
Denna uppgift innebär att skapa ett antal *Factory*-objekt. Dessa ska bygga instanser av de klasser som behövs i spelet vid förfrågan (då motsvarande funktioner anropas). För att uppnå detta kommer de att behöva *Type object*.

Type Object
Objektens typ-data ska separeras från instans-datan. Det innebär att all information som delas av alla instanser av en viss typ (typ-datan) ska sparas i ett eget objekt som sedan kan delas av alla instanser av samma typ.

Exempel: Alla vapen innehåller samma typ av information: de modifierar egenskaper och de har ett namn. Alla vapen av typen *Dagger* delar samma egenskaps-modifierare och samma namn.
Då skapar vi ett *Type object* för alla vapen som kan innehålla de egenskaper som modifieras och hur mycket samt ett namn. För *Dagger* skapar vi en instans av detta *Type object* där vi fyller i de värden som gäller för alla instanser av *Dagger*.

Type object ägs av den *Factory* som ansvarar för att skapa objekt av den typen.

Factory
Vi behöver två *Factory*-objekt (dvs. ni får inte ha flera instanser av en factory):

- EnemyFactory: skapar fiender
- ItemFactory: skapar items

Varje *Factory* ska ha en enum med varje typ som går att skapa.
Båda ska ha en funktion som heter *Create* som skapar och returnerar ett objekt av den önskade typen.

Objektet som returneras ska använda sig av type-objekt datan för den datan som är gemensam för alla objekt av den typen.
Om ni har jobbat rätt så behöver ni inte ändra någon kod som faktiskt använder

sig av items eller enemies, utan bara i funktionerna som anropas på dom. Det är en av fördelarna med enkapsulering (gömma hur klasser fungerar).

Factory-objekt har ofta beroenden av varandra genom pekare eller referenser. I detta fall har EnemyFactory en relation till ItemFactory då fiender kan tappa items.

Kodkrav

- Föremål och fiender ska skapas via olika *Factory*-objekt. Det innebär två olika *Factory*-klasser: EnemyFactory och ItemFactory.
- Föremål och fiender ska innehålla referenser till *Type object*-instanser som ligger i den *Factory* som skapade dem, så att de delar den information som inte ändras och informationen är då densamma för alla objekt av en viss typ.
- Create-funktionerna ska endast ta in en parameter: en enum som anger vilket föremål som ska skapas.
- Endast **en** instans av varje factory får finnas, skicka en referens om den behövs på fler ställen.
- Inga globala/static variabler som inte är const eller constexpr
- Inga publika medlemsvariabler.

Inlämning

Gör mapp med uppgiftens namn. I denna mapp skapar ni ytterligare tvåmappar. En som heter **Exe** och en som heter **Source**.

I **Source**-mappen innehåller all källkod samt projekt filer.
Exe-mappen ska endast filer som behövs för att köra programmet ligga. Ta bort alla skräpfiler!!

Läs igenom ”Inlämningsinstruktioner”-dokumentet för en mer komplett genomgång om hur uppgifter skall lämnas in.

Verktyg

För att lösa uppgiften är följande verktyg, eller koncept tillåtna. En del av er kanske redan har kunskaper i C++ och vet andra sätt att lösa uppgiften med hjälp av andra koncept. Det är inte tillåtet, uppgiften kan och skall lösas endast med angivna verktyg!

```
#include, #pragma once
assert
funktioner
namespace
char, short, int, long, float, double, bool, void, unsigned
std::string, std::string_view, std::vector, std::bitset, iteratorer
class, struct, public, private
const, static
enum class
std::array, referenser, pekare, std::unique_ptr<>, std::span<T>
std::shared_ptr<>, std::weak_ptr<>, std::make_unique<>,
std::make_shared<>
typedef
if, else, else if, switch, while, do, for, return, break
random_device, mt19937, uniform_int_distribution
cout, cin
static_cast<>, const_cast<>
system(“cls”), system(“pause”)
```